

Trabajo Práctico 2:

Deep Q-Network

Lenguajes permitidos: Python 3

Librerías permitidas: PyTorch, Gymnasium $\geq 1.0.0$, NumPy, tqdm

Entrega: A través del campus virtual — informe (máx. 3 páginas en PDF) y código fuente (comprimido en un archivo .zip)

Objetivo

El objetivo principal de este trabajo práctico es implementar el algoritmo [DQN](#) y emplearlo para entrenar agentes en distintos entornos. Como introducción al tema, se solicitará previamente una implementación del algoritmo [Q-learning](#). El trabajo se divide en cuatro partes.

Parte 1 — Entornos de Prueba

Para validar la correcta implementación del agente y de su función de valor, se proponen tres entornos artificiales simples, distintos de los del TP1, cada uno diseñado para aislar aspectos específicos del aprendizaje y permitir una verificación paso a paso del funcionamiento del algoritmo. Cada entorno debe implementarse como una subclase de [gymnasium.Env](#). Se utilizará un factor de descuento $\gamma=0.99$ en todos los casos.

ConstantRewardEnv

Acción única, sin observación, un solo paso, recompensa constante.

- **Acciones disponibles:** 1 (única acción)
- **Observaciones:** constante 0
- **Duración:** 1 paso de tiempo (episodio de una sola transición)
- **Recompensa:** +1 en cada episodio

Este entorno permite verificar si la función de valor aprende correctamente un valor constante. Dado que la observación es siempre la misma y la recompensa también, el agente debería rápidamente aprender que el valor esperado es 1. Si no lo hace, el problema podría estar en el cálculo de la función de pérdida de valor o en la configuración del optimizador.

RandomObsBinaryRewardEnv

Acción única, observación aleatoria, un solo paso, recompensa dependiente de la observación.

- **Acciones disponibles:** 1 (única acción)
- **Observaciones:** aleatorias, con valor +1 o -1
- **Duración:** 1 paso de tiempo
- **Recompensa:** coincide con la observación (+1 o -1)

Este entorno permite evaluar si el agente es capaz de aprender un valor que depende de la observación. A diferencia del entorno anterior, la red debe aprender a predecir un valor condicional a la entrada. Si el agente aprendió correctamente en el entorno **ConstantRewardEnv** pero falla aquí, puede indicar un problema en el flujo de gradientes o en la retropropagación dentro de la red neuronal.

TwoStepDelayedRewardEnv

Acción única, observación determinista, dos pasos, recompensa diferida.

- **Acciones disponibles:** 1 (única acción)
- **Observaciones:** en el primer paso se observa 0; en el segundo paso se observa 1
- **Duración:** 2 pasos por episodio
- **Recompensa:** 0 en el primer paso, +1 al final del episodio

Este entorno introduce un componente temporal: la recompensa no es inmediata, sino que se recibe al final del episodio. El valor correcto debe descontar adecuadamente la recompensa futura. Si el agente aprende correctamente en el entorno **RandomObsBinaryRewardEnv** pero falla en este, el problema puede estar en la implementación del mecanismo de descuento o en el manejo de la acumulación de recompensas a lo largo del episodio.

Parte 2 — Implementación del Algoritmo Q-Learning

Q-learning es un algoritmo de aprendizaje por refuerzo *model-free* dado que permite aprender una política óptima para un agente cuya probabilidad de transición (dinámica) es desconocida, siempre y cuando el problema pueda modelarse como un proceso de decisión de Markov (**MDP**). Su objetivo es aprender una función $Q(s,a)$ que estime el valor esperado de realizar una acción a en un estado s , y seguir luego una política que maximice dicha función.

Durante la interacción con el entorno, el agente observa transiciones de la forma (s,a,r,s') y actualiza su estimación de $Q(s,a)$. El pseudocódigo es el siguiente:

Algorithm 1 Q-learning Pseudocode

- 1: Initialize $Q(s, a)$ for all states s and actions a , to zero.
 - 2: Initialize hyperparameters: learning rate $\alpha \in (0, 1]$, discount factor $\gamma \in [0, 1]$, and exploration rate $\epsilon \in [0, 1]$.
 - 3: **repeat** for each episode:
 - 4: Initialize the agent's state s by sampling from the set of possible initial states.
 - 5: **repeat** for each step of the episode:
 - 6: Choose action a from state s using an ϵ -greedy policy:
 - 7: With probability ϵ , choose a random action a
 - 8: With probability $1 - \epsilon$, choose $a = \arg \max_{a'} Q(s, a')$
 - 9: Take action a , observe reward r and new state s'
 - 10: Update the Q-value for the state-action pair (s, a) :
 - 11: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
 - 12: Update the current state to the new state: $s \leftarrow s'$
 - 13: **until** s is a terminal state or termination criteria is met (e.g., a fixed number of steps)
 - 14: **until** termination criterion is met (e.g., a fixed number of episodes, or convergence)
-

Se pide:

1. Implementar el algoritmo 1 y aplicarlo para entrenar a un agente que aprenda a resolver el entorno [FrozenLake-v1](#), utilizando una grilla 4x4, con una tabla $Q[s,a]$ y su correspondiente regla de actualización tal como se indica en el pseudocódigo.

2. Entrenar al agente durante una cantidad fija de episodios (por ejemplo, 5.000). Experimentar con diferentes hiperparámetros y graficar los rewards, realizando una comparación.
3. Evaluar el desempeño del agente después del entrenamiento:
 - a. Porcentaje de éxito en 100 episodios.
 - b. Porcentaje de éxito en 100 episodios con exploración fija (por ej, $\epsilon = 0.1$).
4. Entrenar con una política de exploración **epsilon-greedy** con opción de decaimiento de ϵ .

```
min_epsilon + (max_epsilon-min_epsilon)*np.exp(-decay_rate*episode)
```

5. Para el punto anterior, producir un gráfico de evolución de la recompensa promedio cada N episodios (por ejemplo, cada 100)
6. Generar un vídeo de evaluación del agente entrenado, mostrando al menos un episodio completo ejecutando su política final de manera determinista (sin exploración, $\epsilon = 0$).

Parte 3 — Implementación del Algoritmo DQN

Antes de comenzar a escribir código, les recomendamos enfáticamente que lean y comprendan el artículo sobre [DQN](#).

El algoritmo **Deep Q-Network (DQN)** utiliza redes neuronales profundas para resolver problemas de decisión secuencial en entornos complejos. Su objetivo es aprender una política que maximice la recompensa esperada acumulada, aproximando la **función Q** con una red neuronal en lugar de una tabla explícita, como en el Q-learning tradicional.

Para lograr una estabilidad adecuada durante el entrenamiento, DQN incorpora dos técnicas clave: **replay buffer**, que almacena experiencias pasadas para romper la correlación temporal de los datos, y **target networks**, que estabilizan las actualizaciones del valor Q.

Algorithm 2: Deep Q-learning con repetición de experiencias

Input: Capacidad del buffer N , número máximo de episodios M , número máximo de pasos T

Output: Función de valor-acción entrenada Q

```
1 Inicializar buffer  $D$  con capacidad  $N$ ;  
2 Inicializar función de valor-acción  $Q$  con pesos aleatorios  $\theta$ ;  
3 Inicializar función de valor-acción objetivo  $Q'$  con pesos  $\theta' = \theta$ ;  
4 for  $\text{episodio} = 1, \dots, M$  do  
5   Inicializar estado  $s_1$ ;  
6   for  $t = 1, \dots, T$  do  
7     Con probabilidad  $\varepsilon$  seleccionar una acción aleatoria  $a_t$ ;  
8     En otro caso seleccionar  $a_t = \arg \max_a Q(s_t, a; \theta)$ ;  
9     Ejecutar acción  $a_t$  en el entorno y observar la recompensa  $r_t$  y estado próximo  $s_{t+1}$ ;  
10    Almacenar la transición  $(s_t, a_t, r_t, s_{t+1})$  en  $D$ ;  
11    Muestrear un minibatch aleatorio de  $K$  transiciones  $(s_j, a_j, r_j, s_{j+1})$  desde  $D$ ;  
12    Definir  
        
$$y_j = \begin{cases} r_j & \text{si el episodio termina en el paso } j + 1 \\ r_j + \gamma \max_{a'} Q'(s_{j+1}, a'; \theta') & \text{en otro caso} \end{cases}$$
  
13    Realizar un paso de descenso de gradiente sobre  
        
$$(y_j - Q(s_j, a_j; \theta))^2$$
  
        con respecto a los parámetros de la red  $\theta$ ;  
14    Cada  $C$  pasos actualizar  $Q' = Q$ ;  
15  end  
16 end
```

3.1 Implementación sobre [CartPole-v1](#)

- Se pide implementar el **Algoritmo 2 con exploración epsilon-greedy con opción de decaimiento de ε** para entrenar un agente que aprenda a resolver el entorno [CartPole-v1](#).
 - La política del agente debe estar representada mediante una red neuronal. Se sugiere comenzar con la siguiente arquitectura base:

```
nn.Linear(input, 64),  
nn.RELU(),  
nn.Linear(64, 64),  
nn.RELU(),  
nn.Linear(64, output)
```

Sugerencia: Pueden experimentar con otras configuraciones de capas (más profundas o más anchas) y distintas funciones de activación (por ejemplo, Tanh, ELU o LeakyReLU) para observar cómo influyen en la velocidad y estabilidad del aprendizaje.

- Entrenar al agente durante una cantidad fija de episodios (por ejemplo, 500 o 1000). Considerar el preprocesado como la función identidad.
- Visualizar el desempeño del entrenamiento utilizando **TensorBoard o similar**, incluyendo al menos:
 - La evolución de la **función de pérdida (loss)**.
 - La **recompensa promedio por episodio**
- Entrenar hasta que se considere el entorno resuelto, es decir, cuando la recompensa converja y se esté logrando el objetivo del entorno. En el caso de CartPole, esto implica mantener el péndulo equilibrado durante todo el episodio (~500 pasos).
- Generar un vídeo de evaluación del agente entrenado, mostrando al menos un episodio completo ejecutando su política final de manera determinista (sin exploración, $\epsilon = 0$).

Hiperparámetros de referencia:

episodes = 500

capacity = 10_000

batch_size = 64

gamma = 0.99

lr = 1e-3

min_eps = 0.05

max_eps = 1.0

target_update_freq = 500

3.2 Competencia de Agentes DQN

¿Recuerdan la competencia de FIA? Volvió. En este apartado se propone una instancia competitiva entre los agentes desarrollados por ustedes, con el objetivo de comparar su desempeño en términos de calidad de la política aprendida.

Deberán entrenar un agente utilizando el algoritmo DQN sobre [MountainCar-v0](#).

A diferencia del punto anterior, en esta sección se busca fomentar la exploración de mejoras sobre el algoritmo base.

Sugerencia: Pueden modificar libremente distintos componentes del agente, incluyendo (pero no limitado a):

- Arquitectura de la red neuronal (profundidad, ancho, funciones de activación)
- Estrategia de exploración (epsilon-greedy, scheduling, alternativas)
- Tamaño y estrategia del replay buffer
- Frecuencia de actualización de la target network
- Función de pérdida y optimizador

- Preprocesamiento de estados
- Modificaciones sobre el entorno (por ejemplo, reward shaping o cambios en la representación del estado)

El objetivo será lograr un agente que **aprenda la mejor política posible**.

Criterios de evaluación

- Se otorgará un **bonus adicional** a aquellos agentes que logren igualar el desempeño de la implementación de referencia desarrollada por Nico.

Evaluación comparativa

Luego de la fecha de entrega, se realizará una evaluación conjunta en la cual todos los modelos serán ejecutados bajo las mismas condiciones experimentales. Para ello, deben entregar los **pesos entrenados de la red neuronal** (en formato **.pt**, incluyendo la arquitectura).

Parte 4 — Experimentos y Evaluación

- Grafique curvas de recompensa vs episodios para DQN vs el REINFORCE del tp pasado para el ambiente **CartPole-v1**
- Grafique curvas de recompensa vs *wall time* para DQN vs el REINFORCE del tp pasado para el ambiente **CartPole-v1**

Entrega

La entrega se realiza a través del campus virtual, e incluye:

Código fuente

- Todo el código utilizado para definir entornos, entrenar agentes y generar las figuras. Debe ser original y estar comprimido en un archivo **.zip**.
- No se evaluará el estilo ni la legibilidad del código, pero se utilizará para comprobar errores de implementación y verificar la autoría de dicho código.
- Los **pesos entrenados de la red neuronal** (en formato **.pt** incluyendo la arquitectura) del punto 3.2.

Informe:

Un archivo PDF de **máximo 3 páginas**, que debe incluir todos los gráficos y responder de manera muy **sintética y clara** las siguientes preguntas:

1. Se incluyen las curvas de **recompensa / pasos_totales** por episodio para cada uno de los tres entornos (para ambos algoritmos):
 - a. ConstantRewardEnv
 - b. RandomObsBinaryRewardEnv
 - c. TwoStepDelayedRewardEnv

Adicionalmente, se debe incluir un gráfico de la evolución de todos los Q-values estimados por el agente para todos los estados a lo largo del entrenamiento. En particular, se espera analizar si dichos valores convergen al valor teórico esperado en cada entorno (por ejemplo, a 1 en ConstantRewardEnv).

2. ¿Qué ventajas y desventajas observó entre el método de Policy Gradient (REINFORCE) y DQN? Considerar aspectos como: estabilidad, velocidad de convergencia, sensibilidad a hiperparámetros, variabilidad de resultados, uso de memoria, etc.
3. ¿Cuál de los dos métodos logró una política óptima (reward 500) en menos tiempo para **CartPole-v1** en sus experimentos? Justifique en base a las curvas de recompensa y comportamiento observado. Incluya los gráficos de la sección 4.
4. Para el caso de **Competencia de Agentes DQN**: ¿Qué modificaciones implementó sobre el algoritmo DQN base, cuál fue la motivación detrás de las mismas y qué impacto tuvieron en el desempeño del agente? En caso de haber realizado cambios sobre el entorno (por ejemplo, reward shaping), descríbalos y justifique su efecto.
5. ¿Qué dificultades encontró al implementar y entrenar cada uno de los algoritmos? Pueden referirse a temas como: debugging, elección de hiperparámetros, uso de TensorBoard, etc.

Criterio de aprobación

El trabajo se considera aprobado si se completa hasta la Parte 3.1